

Здесь будет титульник, листай ниже

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ.....	6
1.1 Описание входных данных.....	8
1.2 Описание выходных данных.....	9
2 МЕТОД РЕШЕНИЯ.....	11
3 ОПИСАНИЕ АЛГОРИТМОВ.....	14
3.1 Алгоритм конструктора класса cl_base.....	14
3.2 Алгоритм метода set_name класса cl_base.....	14
3.3 Алгоритм метода get_name класса cl_base.....	15
3.4 Алгоритм метода get_head_object класса cl_base.....	16
3.5 Алгоритм метода get_subordinate_count класса cl_base.....	16
3.6 Алгоритм метода print_tree класса cl_base.....	16
3.7 Алгоритм метода get_subordinate_by_name класса cl_base.....	17
3.8 Алгоритм метода get_subordinate_by_index класса cl_base.....	18
3.9 Алгоритм деструктора класса cl_base.....	19
3.10 Алгоритм конструктора класса cl_1.....	19
3.11 Алгоритм конструктора класса cl_application.....	19
3.12 Алгоритм метода build_tree_objects класса cl_application.....	20
3.13 Алгоритм метода exec_app класса cl_application.....	22
3.14 Алгоритм функции main.....	24
4 БЛОК-СХЕМЫ АЛГОРИТМОВ.....	25
5 КОД ПРОГРАММЫ.....	41
5.1 Файл cl_1.cpp.....	41
5.2 Файл cl_1.h.....	41
5.3 Файл cl_application.cpp.....	42
5.4 Файл cl_application.h.....	45
5.5 Файл cl_base.cpp.....	45

5.6 Файл cl_base.h.....	47
5.7 Файл main.cpp.....	48
6 ТЕСТИРОВАНИЕ.....	49
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	51

1 ПОСТАНОВКА ЗАДАЧИ

Для организации иерархического построения объектов необходимо разработать базовый класс, который содержит функционал и свойства для построения иерархии объектов. В последующем, в приложениях использовать этот класс как базовый для всех создаваемых классов. Это позволит включать любой объект в состав дерева иерархии объектов.

Каждый объект на дереве иерархии имеет свое место и наименование. Не допускается для одного головного объекта одинаковые наименования в составе подчиненных объектов.

Создать базовый класс со следующими элементами:

- свойства:
 - о наименование объекта (строкового типа);
 - о адрес головного объекта для текущего объекта (для корневого объекта значение адреса равен nullptr);
 - о динамический массив адресов объектов, подчиненных к текущему объекту в дереве иерархии.
- функционал:
 - о параметризированный конструктор с параметрами: указатель на объект базового класса, содержащий адрес головного объекта в дереве иерархии; строкового типа, содержащий наименование создаваемого объекта (имеет значение по умолчанию);
 - о метод редактирования имени объекта. Один параметр строкового типа, содержит новое наименование объекта. Если нет дуближа имени подчиненных объектов у головного, то редактирует имя и возвращает «истину», иначе возвращает «ложь»;
 - о метод получения имени объекта;

- о метод получения адреса головного объекта текущего объекта;
- о метод получения количества подчиненных объектов текущего объекта;
- о метод вывода наименований объектов в дереве иерархии слева направо и сверху вниз;
- о метод получения адреса непосредственно подчиненного объекта по его имени. Если объект не найден, то возвращает nullptr. Один параметр строкового типа, содержит наименование искомого подчиненного объекта;
- о метод получения адреса непосредственно подчиненного объект по его индексу. Допустимые значения индекса от 1 до количества подчиненных объектов. Индексация подчиненных объектов выстраивается согласно порядку добавления головному объекту. Если значение индекса не принадлежит данному интервалу, то возвращает nullptr. У метода один параметр целого типа, содержит значение индекса подчиненного объекта;
- о метод деструктора, который удаляет все объекты на ветке текущего объекта..

Для построения дерева иерархии объектов в качестве корневого объекта используется объект приложение. Класс объекта приложения наследуется от базового класса. Объект приложение реализует следующий функционал:

- метод построения исходного дерева иерархии объектов (конструирования моделируемой системы);
- метод запуска приложения (начало функционирования системы, выполнение алгоритма решения задачи).

Написать программу, которая последовательно строит дерево иерархии объектов, слева направо и сверху вниз. Переход на новый уровень происходит

только от правого (последнего) объекта предыдущего уровня. Для построения дерева использовать **объекты двух производных классов**, наследуемых от базового. Исключить создание объекта если его наименование совпадает с именем уже имеющегося подчиненного объекта у предполагаемого головного. Исключить добавление нового объекта, не последнему подчиненному предыдущего уровня.

Построчно, по уровням вывести наименования объектов построенного иерархического дерева.

Поменять наименования объектов на уровнях иерархии выбранных по их индексам. Если номер уровня или индекса заданы некорректно, то редактирование не выполняется. После редактирования имен построчно, по уровням вывести наименования объектов иерархического дерева.

Основная функция должна иметь следующий вид:

```
int main ( )
{
    cl_application  ob_cl_application ( nullptr ); // создание корневого
    объекта
    ob_cl_application.build_tree_objects ( );          // конструирование
    системы, построение дерева объектов
    return ob_cl_application.exec_app ( );             // запуск системы
}
```

Наименование класса `cl_application` и идентификатора корневого объекта `ob_cl_application` могут быть изменены разработчиком.

Все версии курсовой работы имеют такую основную функцию.

В данной версии курсовой работы в методе `build_tree_objects ( )` выполняется только построение дерева иерархии объектов.

1.1 Описание входных данных

Первая строка:

«имя корневого объекта»

Вторая строка и последующие строки:

«имя головного объекта» «имя подчиненного объекта»

Создается подчиненный объект и добавляется в иерархическое дерево. Если «имя головного объекта» равняется «имени подчиненного объекта», то новый объект не создается и построение дерева объектов завершается.

Следующая строка и последующие строки

«номер уровня» «индекс элемента на уровне» «новое наименование объекта»

Если «номер уровня» равно нулю, то редактирование имен объектов завершается.

Для изменения наименования корневого объекта строка входных данных имеет вид:

1 1 «новое наименование корневого объекта»

Пример ввода:

```
Object_root
Object_root Object_1
Object_root Object_2
Object_root Object_3
Object_3 Object_4
Object_3 Object_5
Object_6 Object_6
1 1 root_Object
3 1 Object_15
4 1 Object_41
2 7 Object_72
2 2 Object_55
0
```

Дерево объектов, которое будет построено по данному примеру:

```
Object_root
  Object_1
  Object_2
  Object_3
    Object_4
    Object_5
```

1.2 Описание выходных данных

Первая строка:

«имя корневого объекта»

Вторая строка и последующие строки имена головного и подчиненных объектов очередного уровня разделенных двумя пробелами.

«имя головного объекта» «имя подчиненного объекта»[[«имя подчиненного объекта»]]

Со следующей строки аналогичный вывод с отредактированными наименованиями объектов.

Пример вывода:

```
Object_root
Object_root Object_1 Object_2 Object_3
Object_3 Object_4 Object_5
root_Object
root_Object Object_1 Object_55 Object_3
Object_3 Object_15 Object_5
```


2 МЕТОД РЕШЕНИЯ

Для решения задачи используется:

- объект `cin` класса `istream` предназначен для ввода с клавиатуры;
- объект `cout` класса `ostream` предназначен для вывода на экран;
- объект `ob_cl_application` класса `cl_application` предназначен для объявления объекта;
- `for` - оператор цикла со счетчиком;
- `if...else` - условный оператор;
- `while` - оператор цикла с предусловием.

Класс `cl_base`:

- свойства/поля:
 - поле имя объекта:
 - наименование — `name`;
 - тип — `string`;
 - модификатор доступа — `protected`;
 - поле указатель головного объекта:
 - наименование — `p_head_object`;
 - тип — `cl_base*`;
 - модификатор доступа — `protected`;
 - поле динамический массив адресов объектов, подчиненных текущему объекту в дереве иерархии:
 - наименование — `v_child_objects`;
 - тип — `vector<cl_base*>`;
 - модификатор доступа — `protected`;
- функционал:
 - метод `cl_base` — конструктор с параметрами;

- о метод `set_name` — метод изменения имени;
- о метод `get_name` — метод получения имени;
- о метод `get_head_object` — метод получения адреса головного объекта;
- о метод `get_subordinate_count` — метод получения количества подчиненных объектов текущего объекта;
- о метод `print_tree` — метод вывода наименований объектов в дереве иерархии слева направо и сверху вниз;
- о метод `get_subordinate_by_name` — метод получения адреса непосредственного подчиненного объекта по его имени;
- о метод `get_subordinate_by_index` — метод получения адреса непосредственного подчиненного объекта по его индексу;
- о метод `~cl_base` — деструктор.

Класс `cl_1`:

- функционал:
 - о метод `cl_1` — конструктор с параметрами.

Класс `cl_application`:

- функционал:
 - о метод `cl_application` — конструктор с параметром;
 - о метод `build_tree_objects` — метод построения исходного дерева иерархии объектов;
 - о метод `exes_app` — метод запуска приложения.

Таблица 1 – Иерархия наследования классов

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
1	<code>cl_base</code>			базовый класс	
		<code>cl_1</code>	<code>public</code>		2

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
		cl_application	public		3
2	cl_1			производный класс	
3	cl_application			класс объекта приложения	

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм конструктора класса `cl_base`

Функционал: конструктор с параметрами.

Параметры: `cl_base* p_head_object` - указатель головного объекта, `string name` - имя объекта.

Алгоритм конструктора представлен в таблице 2.

Таблица 2 – Алгоритм конструктора класса `cl_base`

№	Предикат	Действия	№ перехода
1		присвоение защищенному свойству <code>p_head_object</code> значения параметра <code>p_head_object</code>	2
2		присвоение защищенному свойству <code>name</code> значения параметра <code>name</code>	3
3	<code>p_head_object != nullptr</code>	добавление адреса настоящего объекта в массив <code>v_child_objects</code> головного объекта <code>p_head_object</code>	∅
			∅

3.2 Алгоритм метода `set_name` класса `cl_base`

Функционал: метод изменения имени.

Параметры: `string new_name` - новое имя.

Возвращаемое значение: `bool` - истина или ложь.

Алгоритм метода представлен в таблице 3.

Таблица 3 – Алгоритм метода *set_name* класса *cl_base*

№	Предикат	Действия	№ перехода
1	<code>p_head_object == nullptr</code>	присвоение защищенному свойству <code>name</code> значение параметра <code>new_name</code>	2
			3
2		возвращение истины	∅
3	<code>i < p_head_object->v_child_objects.size()</code>		4
			5
4	<code>p_head_object->v_child_objects[i]->name == new_name</code>	возвращение лжи	∅
			5
5		присвоение защищенному свойству <code>name</code> значение параметра <code>new_name</code>	6
6		возвращение истины	∅

3.3 Алгоритм метода *get_name* класса *cl_base*

Функционал: метод получения имени.

Параметры: нет.

Возвращаемое значение: `string` - имя объекта.

Алгоритм метода представлен в таблице 4.

Таблица 4 – Алгоритм метода *get_name* класса *cl_base*

№	Предикат	Действия	№ перехода
1		возвращение защищенного свойства <code>name</code>	∅

3.4 Алгоритм метода `get_head_object` класса `cl_base`

Функционал: метод получения адреса головного объекта.

Параметры: нет.

Возвращаемое значение: `cl_base*` - адрес указателя головного объекта.

Алгоритм метода представлен в таблице 5.

Таблица 5 – Алгоритм метода `get_head_object` класса `cl_base`

№	Предикат	Действия	№ перехода
1		возвращение защищенного свойства <code>p_head_object</code>	Ø

3.5 Алгоритм метода `get_subordinate_count` класса `cl_base`

Функционал: метод получения количества подчиненных объектов текущего объекта.

Параметры: нет.

Возвращаемое значение: `int` - количество подчиненных объектов.

Алгоритм метода представлен в таблице 6.

Таблица 6 – Алгоритм метода `get_subordinate_count` класса `cl_base`

№	Предикат	Действия	№ перехода
1		возвращение размерности защищенного свойства <code>v_child_objects</code>	Ø

3.6 Алгоритм метода `print_tree` класса `cl_base`

Функционал: метод вывода наименований объектов в дереве иерархии слева направо и сверху вниз.

Параметры: нет.

Возвращаемое значение: `void` - ничего не возвращает.

Алгоритм метода представлен в таблице 7.

Таблица 7 – Алгоритм метода `print_tree` класса `cl_base`

№	Предикат	Действия	№ перехода
1		вывод на экран корневого объекта	2
2		создание объекта очереди <code>q</code> , который состоит из указателей на объекты класса <code>cl_base</code>	3
3		добавление в очередь настоящего объекта	4
4	<code>!q.empty()</code>	инициализация указателя <code>current</code> на объект класса <code>cl_base</code> и присвоение ему значения первого элемента очереди	5
			∅
5		удаление первого элемента очереди	6
6	<code>current->get_subordinate_count() > 0</code>	вывод на экран имени родителя	7
			4
7	<code>i < current->get_subordinate_count()</code>	инициализация указателя <code>child</code> на объект класса <code>cl_base</code> и присвоение ему результата вызова метода <code>get_subordinate_by_index</code> с параметром <code>i</code> от объекта <code>current</code>	8
		перевод на новую строку	4
8	<code>child != nullptr</code>	вывод на экран " " и результата вызова метода <code>get_name</code> от объекта <code>child</code>	9
			7
9		добавление <code>child</code> в очередь	7

3.7 Алгоритм метода `get_subordinate_by_name` класса `cl_base`

Функционал: метод получения адреса непосредственного подчиненного объекта по его имени.

Параметры: `string name` - имя непосредственно подчиненного объекта.

Возвращаемое значение: `cl_base*` - адрес непосредственно подчиненного

объекта.

Алгоритм метода представлен в таблице 8.

Таблица 8 – Алгоритм метода *get_subordinate_by_name* класса *cl_base*

№	Предикат	Действия	№ перехода
1	$i < v_child_objects.size()$		2
			3
2	$v_child_objects[i] \rightarrow name == name$	возвращаем адрес объекта	$\emptyset$
			1
3		возвращаем nullptr	$\emptyset$

3.8 Алгоритм метода *get_subordinate_by_index* класса *cl_base*

Функционал: метод получения адреса непосредственного подчиненного объекта по его индексу.

Параметры: *int index* - индекс.

Возвращаемое значение: *cl_base** - адрес непосредственно подчиненного объекта.

Алгоритм метода представлен в таблице 9.

Таблица 9 – Алгоритм метода *get_subordinate_by_index* класса *cl_base*

№	Предикат	Действия	№ перехода
1	$index < 1 \parallel index > (int)v_child_objects.size()$	возвращение nullptr	$\emptyset$
			2
2		возвращение подчиненного объекта <i>v_child_objects</i> с индексом <i>index-1</i>	$\emptyset$

3.9 Алгоритм деструктора класса cl_base

Функционал: деструктор.

Параметры: нет.

Алгоритм деструктора представлен в таблице 10.

Таблица 10 – Алгоритм деструктора класса cl_base

№	Предикат	Действия	№ перехода
1	i < v_child_objects.size()	удаление элемента массива v_child_objects с индексом i	1
			∅

3.10 Алгоритм конструктора класса cl_1

Функционал: конструктор с параметрами.

Параметры: cl_base* p_head_object - указатель головного объекта, string name - имя объекта.

Алгоритм конструктора представлен в таблице 11.

Таблица 11 – Алгоритм конструктора класса cl_1

№	Предикат	Действия	№ перехода
1			∅

3.11 Алгоритм конструктора класса cl_application

Функционал: конструктор с параметром.

Параметры: cl_base* p_head_object - указатель головного объекта.

Алгоритм конструктора представлен в таблице 12.

Таблица 12 – Алгоритм конструктора класса *cl_application*

№	Предикат	Действия	№ перехода
1			Ø

3.12 Алгоритм метода *build_tree_objects* класса *cl_application*

Функционал: метод построения исходного дерева иерархии объектов.

Параметры: нет.

Возвращаемое значение: *void* - ничего не возвращает.

Алгоритм метода представлен в таблице 13.

Таблица 13 – Алгоритм метода *build_tree_objects* класса *cl_application*

№	Предикат	Действия	№ перехода
1		инициализация переменных <i>root_name</i> , <i>parent_name</i> , <i>child_name</i> типа <i>string</i>	2
2		ввод с клавиатуры переменной <i>root_name</i>	3
3		вызов метода <i>set_name</i> с параметром <i>root_name</i>	4
4		инициализация переменной <i>count</i> типа <i>int</i> со значением, равным 0	5
5	<i>std::cin >> parent_name >> child_name</i>		6
			Ø
6	<i>parent_name == child_name</i>		Ø
			7
7		инициализация указателя <i>parent</i> на объект класса <i>cl_base</i> и присвоение значения <i>nullptr</i>	8
8		создание объекта очереди <i>q</i> , который состоит из указателей на объекты класса <i>cl_base</i>	9
9		добавление в очередь настоящего объекта	10
10	<i>!q.empty()</i>	инициализация указателя <i>current</i> на объект класса	11

№	Предикат	Действия	№ перехода
		cl_base и присвоение ему значения первого элемента очереди	
			15
11		удаление первого элемента очереди	12
12	current->get_name() == parent_name	присвоение указателю parent значения указателя current	13
			14
13		остановка цикла	15
14	i < current->get_subordinate_count()	добавляем в очередь current->get_subordinate_by_index()	14
			10
15	parent == nullptr		5
			16
16		инициализация переменной valid_parent типа bool со значением false	17
17	parent->get_head_object() == nullptr	присвоение переменной valid_parent значения true	22
			18
18	parent->get_subordinate_count() > 0	присвоение переменной valid_parent значения true	22
			19
19		инициализация указателя head на объект класса cl_base и присвоение результата вызова метода get_head_object() от объекта parent	20
20		инициализация переменной count типа int и присвоение результата вызова метода get_subordinate_count() от объекта head	21
21	count > 0 && head->get_subordinate_by_index(co	присвоение переменной valid_parent значения true	22

№	Предикат	Действия	№ перехода
	unt) == parent		
			22
22	!valid_parent		5
			23
23	parent->get_subordinate_by_name(child_name) != nullptr		5
			24
24		динамическое создание объекта класса cl_1 с параметрами parent, child_name	5

3.13 Алгоритм метода exes_app класса cl_application

Функционал: метод запуска приложения.

Параметры: нет.

Возвращаемое значение: int - индикатор корректности завершения программы.

Алгоритм метода представлен в таблице 14.

Таблица 14 – Алгоритм метода exes_app класса cl_application

№	Предикат	Действия	№ перехода
1		вызов метода print_tree	2
2		инициализация переменных level, index типа int	3
3		инициализация переменной new_name типа string	4
4	std::cin >> level		5
			18
5	level == 0		18
		ввод с клавиатуры переменных index и new_name	6
6		инициализация переменной count_lvl типа int	7

№	Предикат	Действия	№ перехода
		значением 1	
7		инициализация указателя current на объект класса cl_base со значением настоящего объекта	8
8		инициализация переменной is_valid типа bool со значением true	9
9	count_lvl < level - 1	присвоение указателю current результата вызова метода get_subordinate_by_index с параметром current->get_subordinate_count() от объекта current	10
			13
10	current == nullptr	присвоение переменной is_valid значения false	11
			12
11		остановка цикла	13
12		увеличение переменной count_lvl на 1	9
13	is_valid == false		4
			14
14	level == 1 && index == 1	вызов метода set_name с параметром new_name от объекта current	4
			15
15		присвоение указателю current результата вызова метода get_subordinate_by_index с параметром index от объекта current	16
16	current == nullptr		4
			17
17		вызов метода set_name с параметром new_name от объекта current	4
18		вызов метода print tree	∅

3.14 Алгоритм функции main

Функционал: основной алгоритм программы.

Параметры: нет.

Возвращаемое значение: int - индикатор корректности завершения программы.

Алгоритм функции представлен в таблице 15.

Таблица 15 – Алгоритм функции main

№	Предикат	Действия	№ перехода
1		инициализация объекта ob_cl_cpplication класса cl_application с параметром nullptr	2
2		вызов метода build_tree_objects от объекта ob_cl_cpplication	3
3		возвращение результата вызова метода exes_app от объекта ob_cl_cpplication	Ø

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-16.

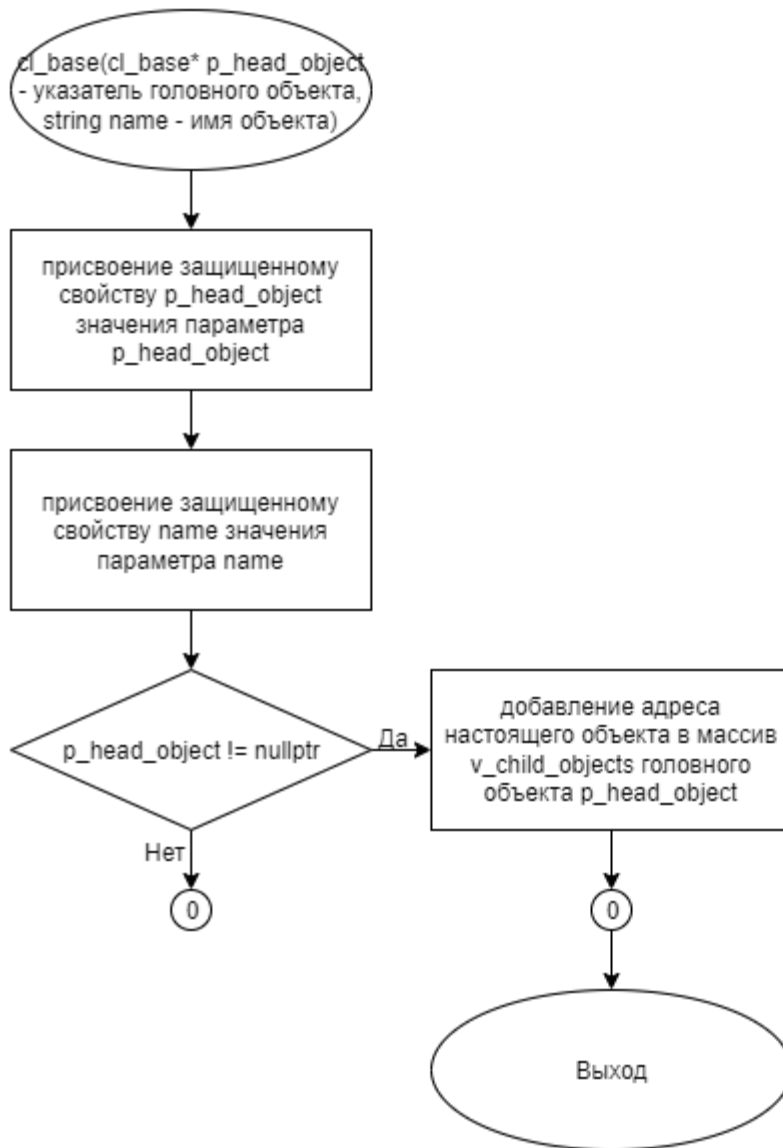


Рисунок 1 – Блок-схема алгоритма

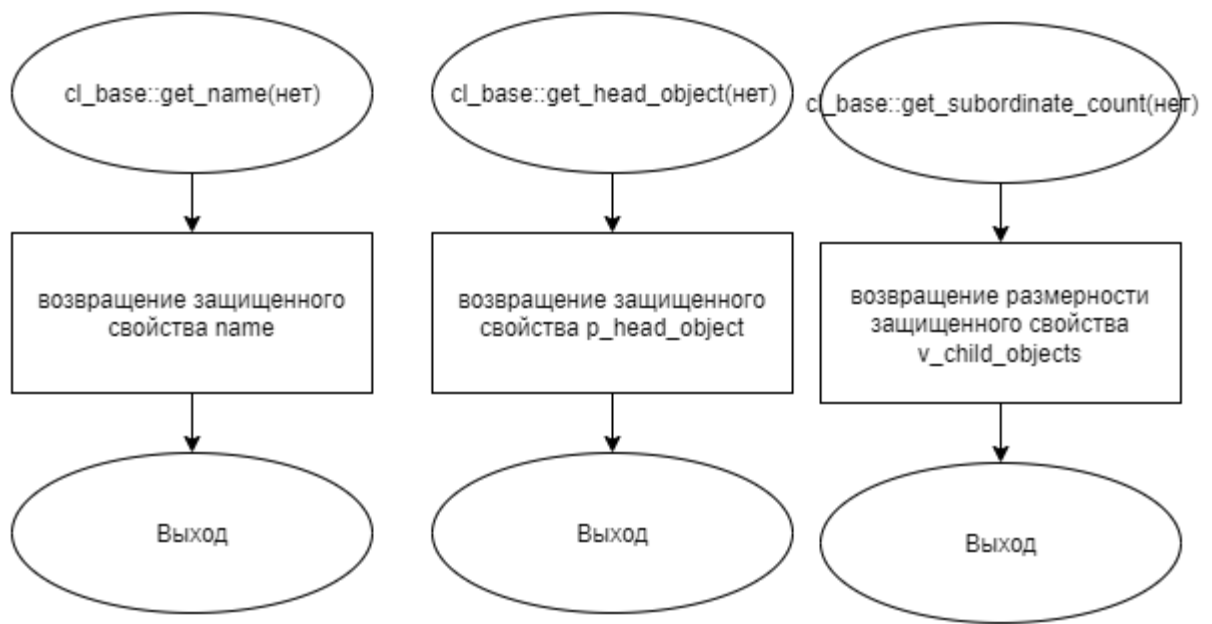


Рисунок 2 – Блок-схема алгоритма

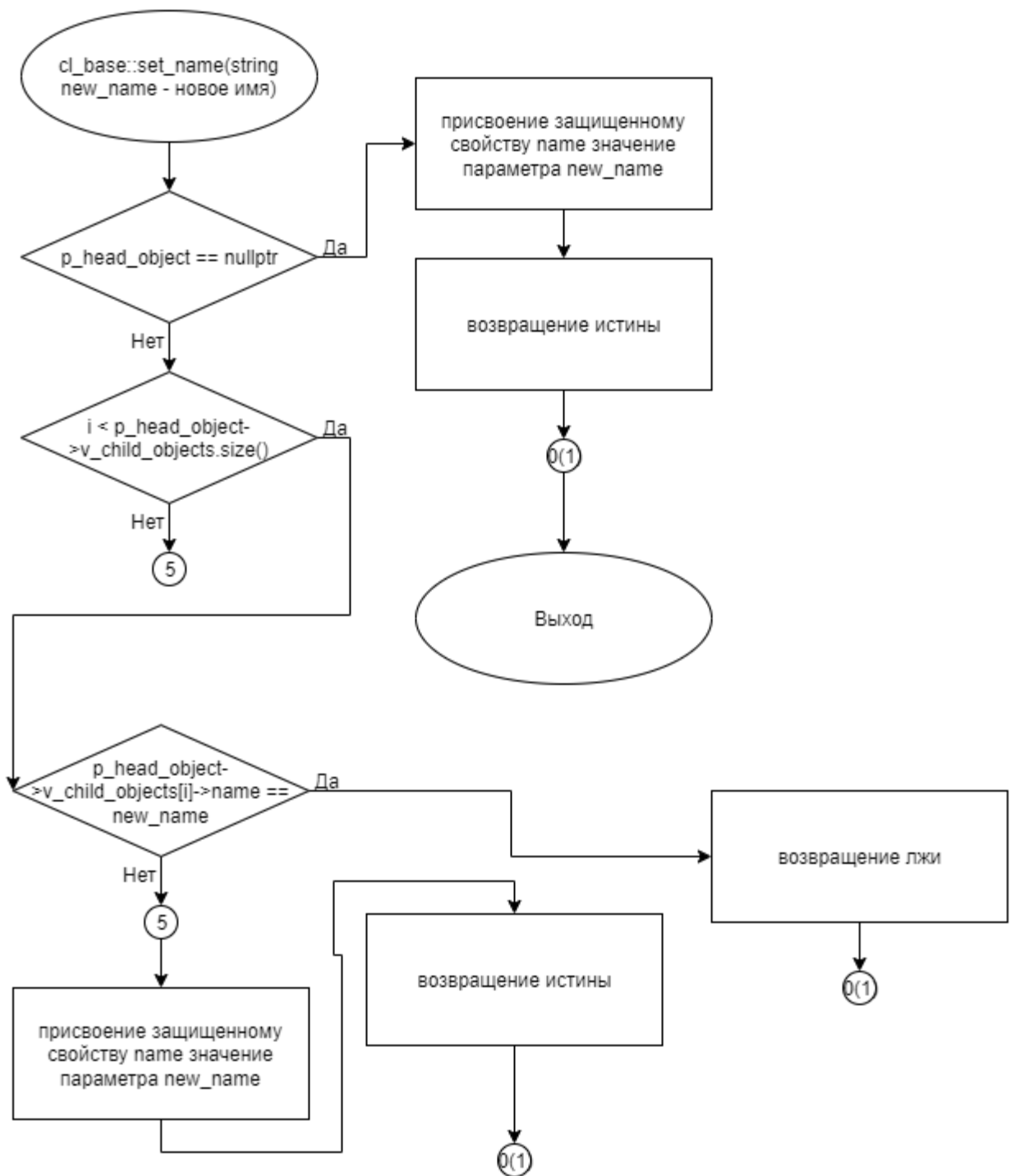


Рисунок 3 – Блок-схема алгоритма

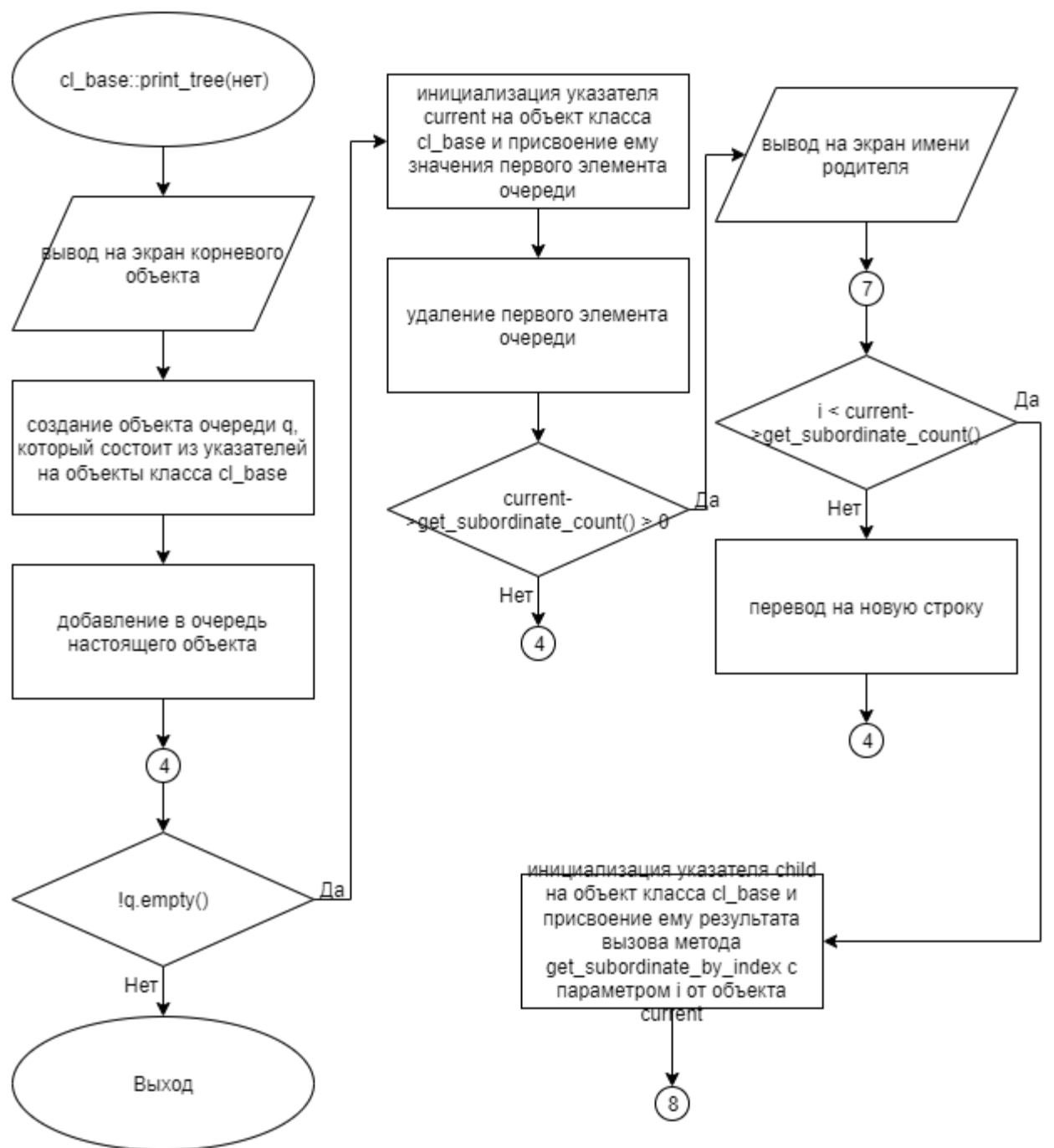


Рисунок 4 – Блок-схема алгоритма

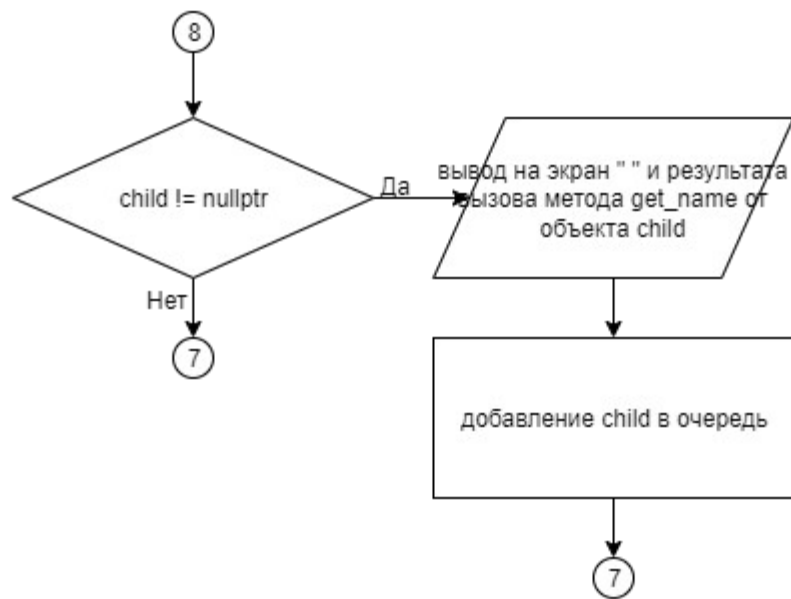


Рисунок 5 – Блок-схема алгоритма

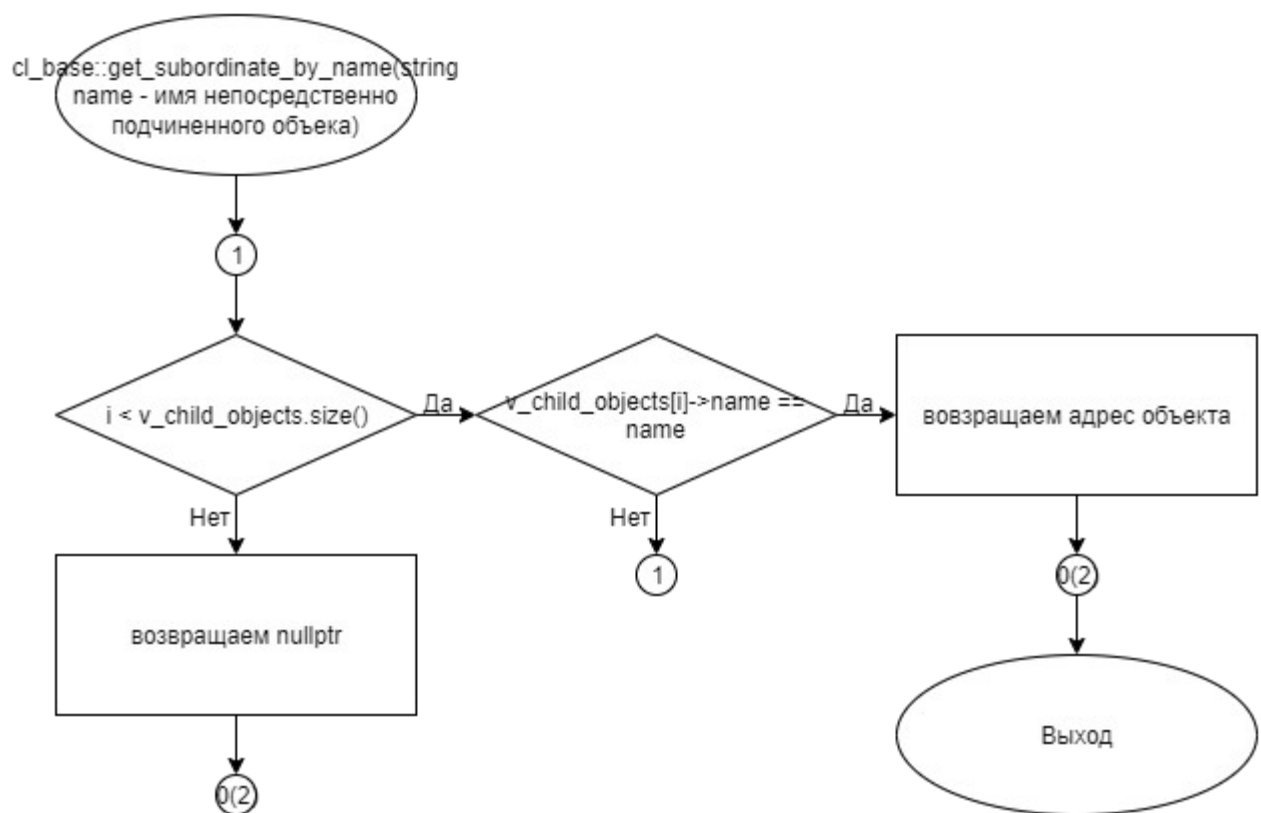


Рисунок 6 – Блок-схема алгоритма

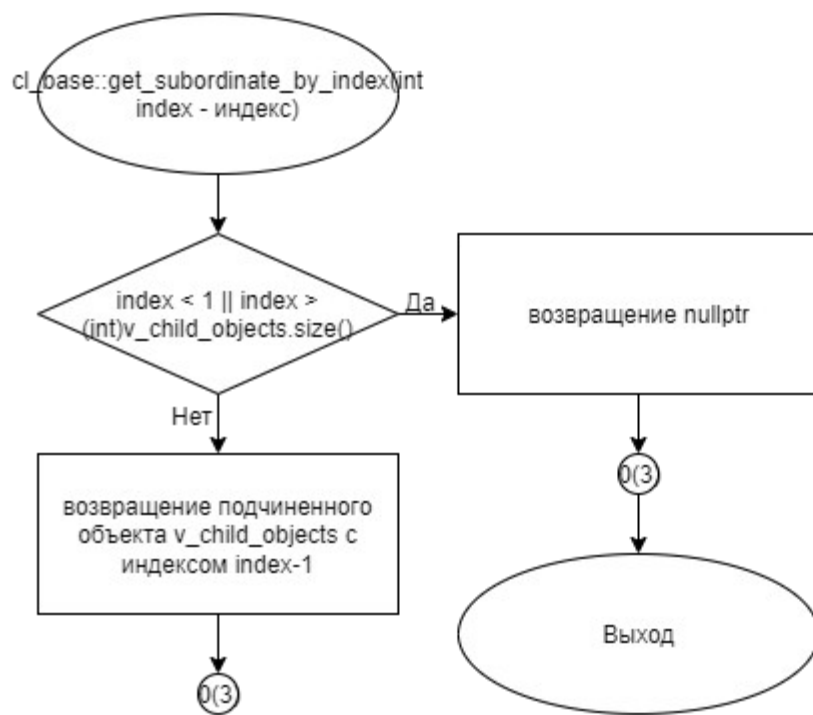


Рисунок 7 – Блок-схема алгоритма

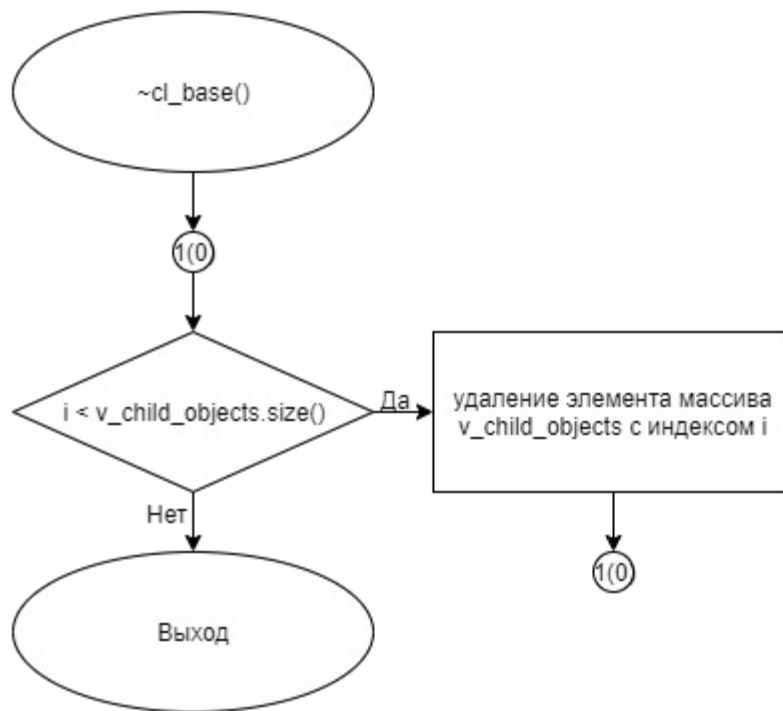


Рисунок 8 – Блок-схема алгоритма



Рисунок 9 – Блок-схема алгоритма

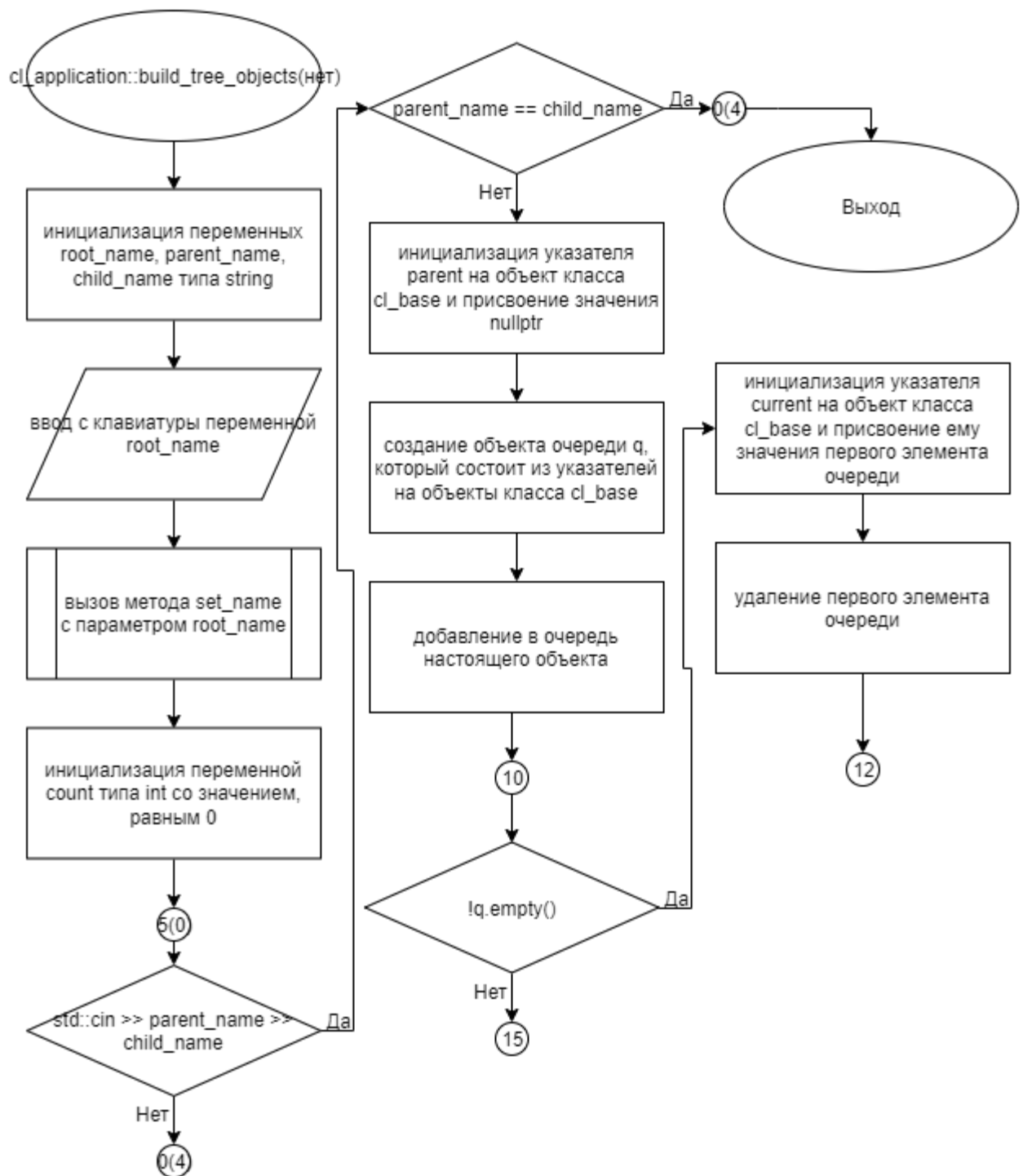


Рисунок 10 – Блок-схема алгоритма

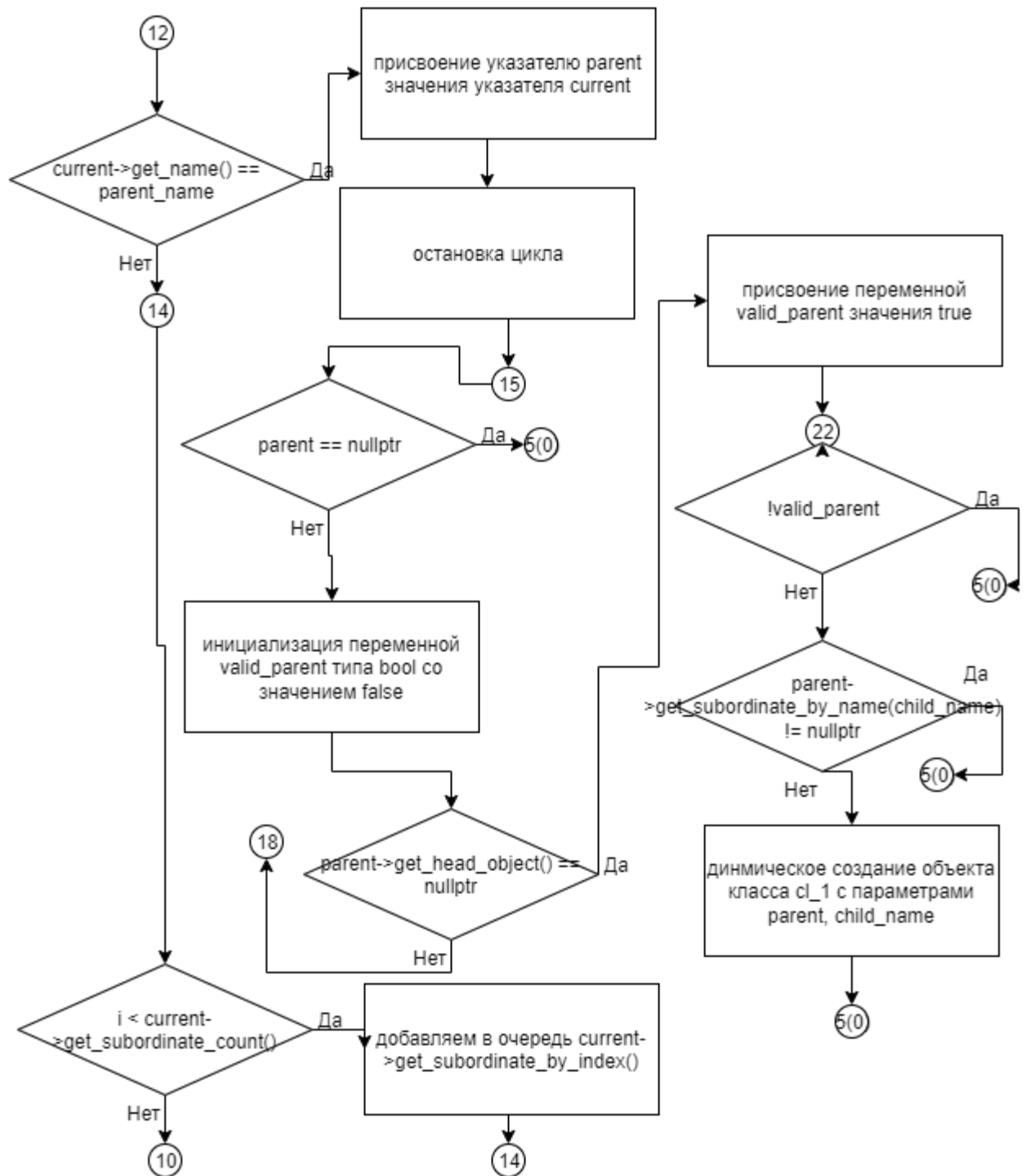


Рисунок 11 – Блок-схема алгоритма

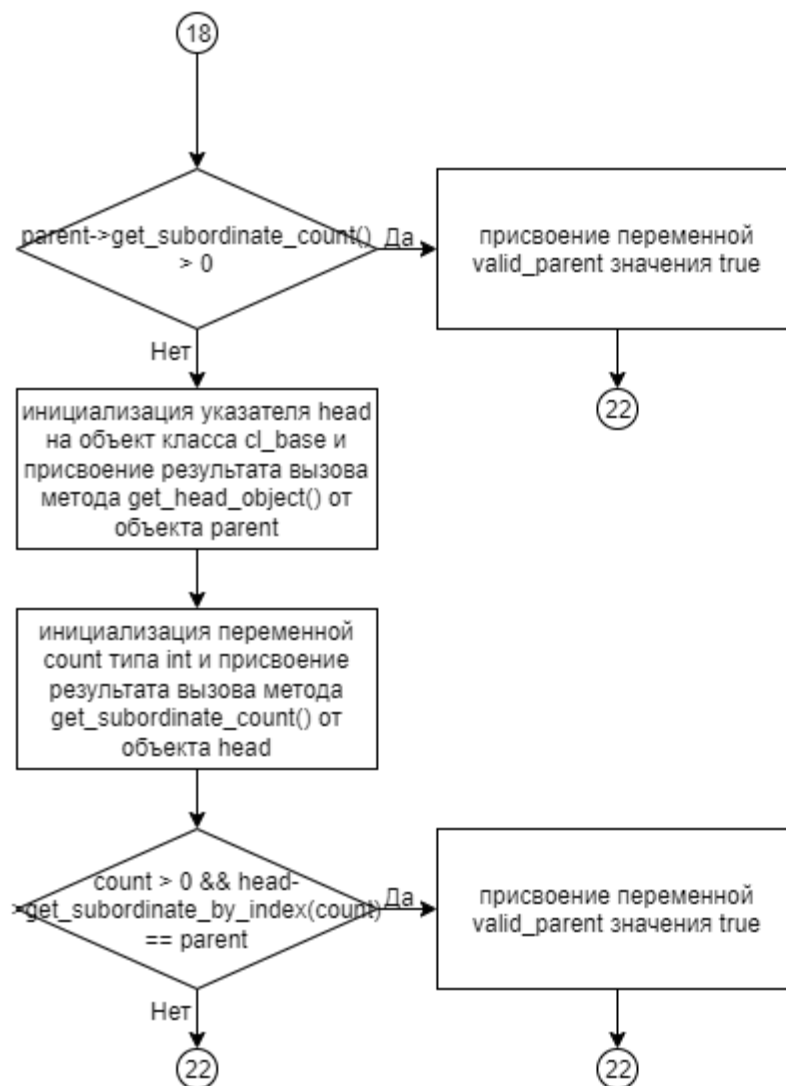


Рисунок 12 – Блок-схема алгоритма

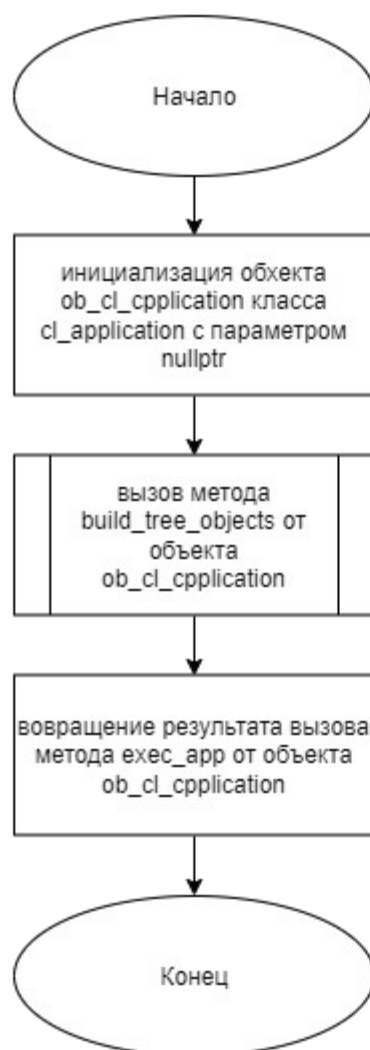


Рисунок 13 – Блок-схема алгоритма

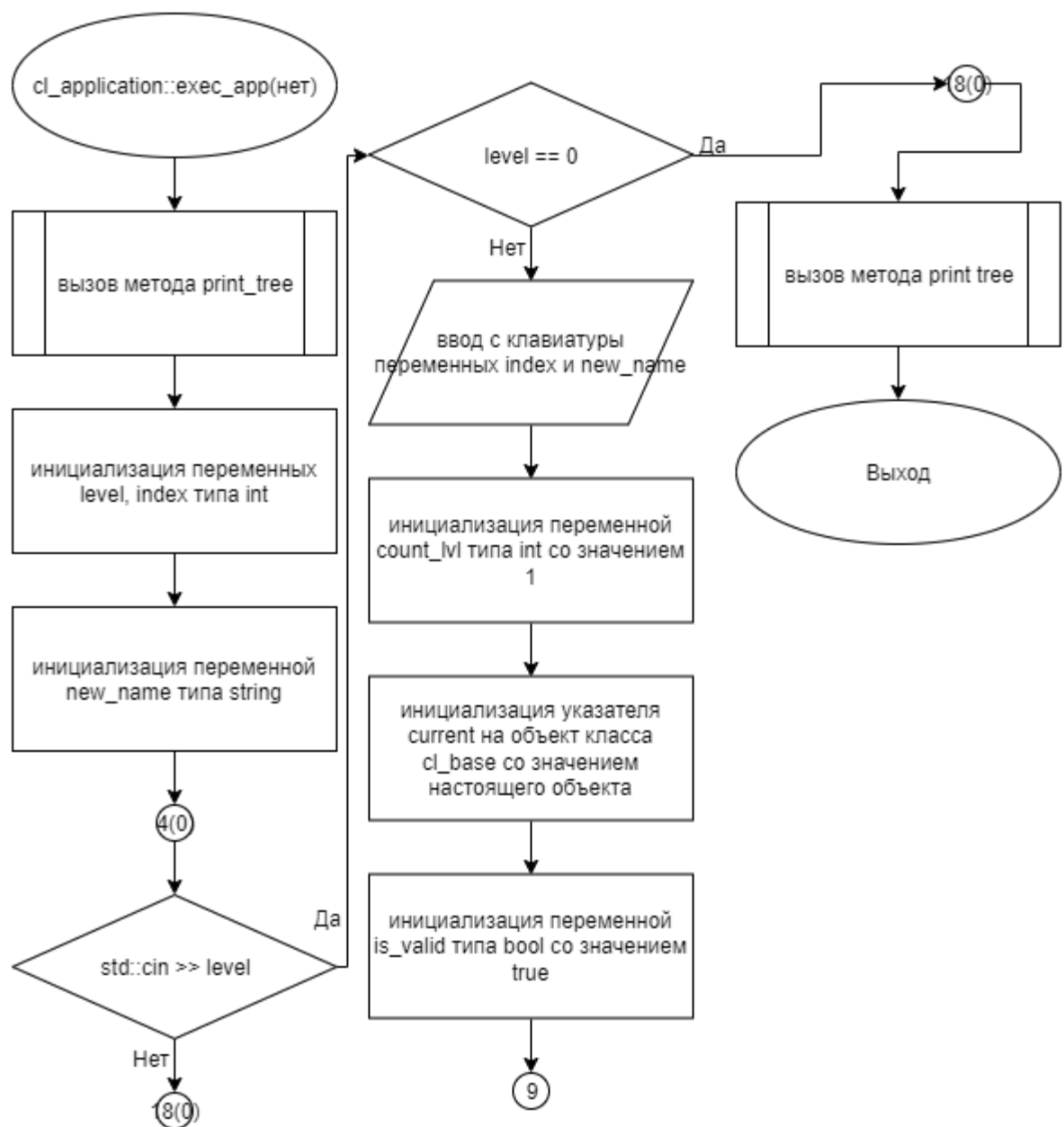


Рисунок 14 – Блок-схема алгоритма

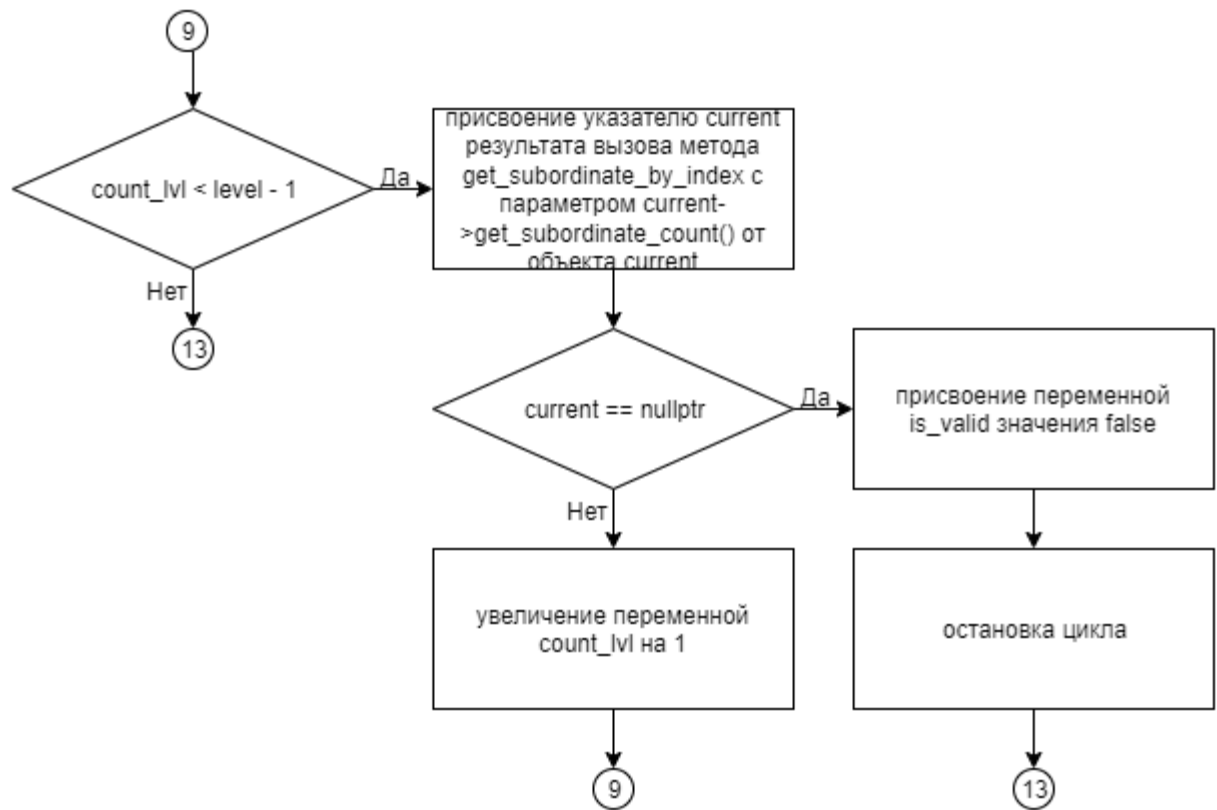


Рисунок 15 – Блок-схема алгоритма

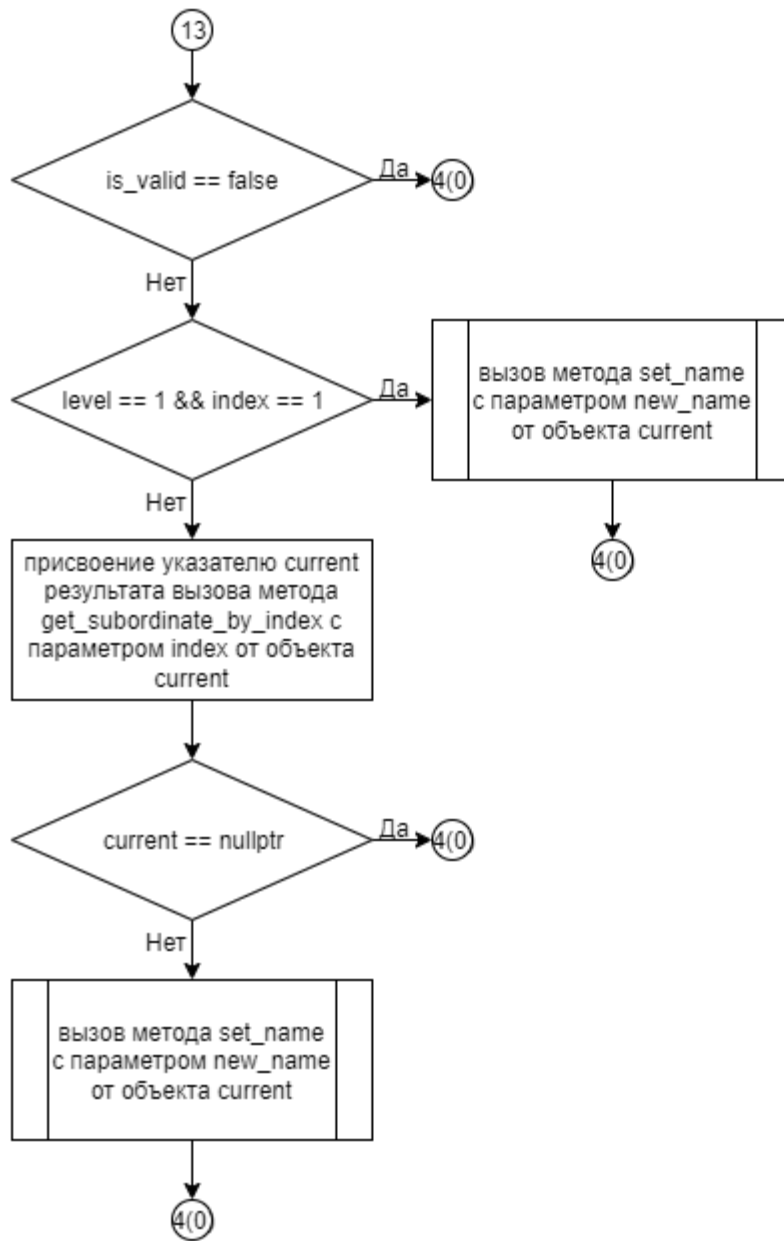


Рисунок 16 – Блок-схема алгоритма

5 КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

5.1 Файл cl_1.cpp

Листинг 1 – cl_1.cpp

```
#include "cl_1.h"
#include "cl_base.h"
#include <string>

cl_1::cl_1(cl_base* p_head_object, std::string name): cl_base(p_head_object,
name)
{
}
```

5.2 Файл cl_1.h

Листинг 2 – cl_1.h

```
#ifndef __CL_1__H
#define __CL_1__H

#include "cl_base.h"
#include <string>

class cl_1 : public cl_base
{
public:
    cl_1(cl_base* p_head_object, std::string name);
};

#endif
```

5.3 Файл cl_application.cpp

Листинг 3 – cl_application.cpp

```
#include "cl_application.h"
#include "cl_1.h"
#include <iostream>
#include <queue>
#include <vector>
#include <string>

cl_application::cl_application(cl_base* p_head_object):
cl_base(p_head_object)
{
}

void cl_application::build_tree_objects()
{
    std::string root_name, parent_name, child_name;
    std::cin >> root_name;
    set_name(root_name);

    while (std::cin >> parent_name >> child_name)
    {
        // break check
        if (parent_name == child_name)
        {
            break;
        }

        cl_base* parent = nullptr;

        std::queue<cl_base*> q;
        q.push(this);

        while (!q.empty())
        {
            cl_base* current = q.front();
            q.pop();

            if (current->get_name() == parent_name)
            {
                parent = current;
                break;
            }

            for (int i = 1; i <= current->get_subordinate_count(); i++)
            {
                q.push(current->get_subordinate_by_index(i));
            }
        }

        // skip non-existent parent
        if (parent == nullptr)
```



```

        {
            continue;
        }

        // check if can be parent
        bool valid_parent = false;

        // root can always be parent
        if (parent->get_head_object() == nullptr)
        {
            valid_parent = true;
        }
        // already has children
        else if (parent->get_subordinate_count() > 0)
        {
            valid_parent = true;
        }
        else
        {
            cl_base* head = parent->get_head_object();
            int count = head->get_subordinate_count();

            // last child
            if (count > 0 && head->get_subordinate_by_index(count) == parent)
            {
                valid_parent = true;
            }
        }

        // check if object can be a parent
        if (!valid_parent)
        {
            continue;
        }

        // check for duplicates in branch
        if (parent->get_subordinate_by_name(child_name) != nullptr)
        {
            continue;
        }

        new cl_1(parent, child_name);
    }
}

int cl_application::exec_app()
{
    print_tree();

    int level, index;
    std::string new_name;

    while (std::cin >> level)
    {
        // break check
    }
}

```

```

        if (level == 0)
        {
            break;
        }

        std::cin >> index >> new_name;

        int count_lvl = 1;
        cl_base* current = this;
        bool is_valid = true;

        // descend to requested level
        while (count_lvl < level - 1)
        {
            current = current->get_subordinate_by_index(current-
>get_subordinate_count());

            if (current == nullptr)
            {
                is_valid = false;
                break;
            }

            count_lvl++;
        }

        if (is_valid == false)
            continue;

        if (level == 1 && index == 1)
        {
            current->set_name(new_name);
            continue;
        }

        current = current->get_subordinate_by_index(index);

        if (current == nullptr)
            continue;

        current->set_name(new_name);
    }

    print_tree();

    return 0;
}

```

5.4 Файл cl_application.h

Листинг 4 – cl_application.h

```
#ifndef __CL_APPLICATION__H
#define __CL_APPLICATION__H

#include "cl_base.h"

class cl_application : public cl_base
{
public:
    cl_application(cl_base* p_head_object);

    void build_tree_objects();
    int exec_app();
};

#endif
```

5.5 Файл cl_base.cpp

Листинг 5 – cl_base.cpp

```
#include "cl_base.h"
#include <string>
#include <queue>
#include <iostream>

cl_base::cl_base(cl_base* p_head_object, std::string name)
{
    this->p_head_object = p_head_object;
    this->name = name;

    if (p_head_object != nullptr)
    {
        p_head_object->v_child_objects.push_back(this);
    }
}

bool cl_base::set_name(std::string new_name)
{
    if (p_head_object == nullptr)
    {
        name = new_name;
        return true;
    }
}
```

```

        for (int i = 0; i < p_head_object->v_child_objects.size(); i++)
        {
            if (p_head_object->v_child_objects[i]->name == new_name)
            {
                return false;
            }
        }

        name = new_name;

        return true;
    }

    std::string cl_base::get_name()
    {
        return name;
    }

    cl_base* cl_base::get_head_object()
    {
        return p_head_object;
    }

    int cl_base::get_subordinate_count()
    {
        return v_child_objects.size();
    }

    cl_base* cl_base::get_subordinate_by_name(std::string name)
    {
        for (int i = 0; i < v_child_objects.size(); i++)
        {
            if (v_child_objects[i]->name == name)
            {
                return v_child_objects[i];
            }
        }

        return nullptr;
    }

    cl_base* cl_base::get_subordinate_by_index(int index)
    {
        if (index < 1 || index > (int)v_child_objects.size())
        {
            return nullptr;
        }

        return v_child_objects[index - 1];
    }

    void cl_base::print_tree()
    {
        std::cout << this->get_name() << std::endl;
    }

```

```

std::queue<cl_base*> q;
q.push(this);

while (!q.empty())
{
    cl_base* current = q.front();
    q.pop();

    if (current->get_subordinate_count() > 0)
    {
        // print parent
        std::cout << current->get_name();

        // print names of all subordinates
        for (int i = 1; i <= current->get_subordinate_count(); i++)
        {
            cl_base* child = current->get_subordinate_by_index(i);
            if (child != nullptr)
            {
                std::cout << " " << child->get_name();
                q.push(child);
            }
        }
        std::cout << std::endl;
    }
}

cl_base::~~cl_base()
{
    for (int i = 0; i < v_child_objects.size(); i++)
    {
        delete v_child_objects[i];
    }
}

```

5.6 Файл cl_base.h

Листинг 6 – cl_base.h

```

#ifndef __CL_BASE__H
#define __CL_BASE__H

#include <string>
#include <vector>

class cl_base
{
protected:
    std::string name;

```

```

    cl_base *p_head_object;
    std::vector<cl_base*> v_child_objects;

public:
    cl_base(cl_base* p_head_object, std::string name = "Base_object");
    bool set_name(std::string new_name);
    std::string get_name();
    cl_base* get_head_object();
    int get_subordinate_count();
    void print_tree();

    // get address by name
    cl_base* get_subordinate_by_name(std::string name);
    // get address by index
    cl_base* get_subordinate_by_index(int index);
    ~cl_base();
};

#endif

```

5.7 Файл main.cpp

Листинг 7 – main.cpp

```

#include <stdlib.h>
#include <stdio.h>
#include "cl_application.h"

int main()
{
    cl_application ob_cl_application(nullptr);
    ob_cl_application.build_tree_objects();
    return ob_cl_application.exec_app();
}

```

6 ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 16.

Таблица 16 – Результат тестирования программы

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
Object_root Object_root Object_1 Object_root Object_2 Object_root Object_3 Object_3 Object_4 Object_3 Object_5 Object_6 Object_6 1 1 root_Object 3 1 Object_15 4 1 Object_41 2 7 Object_72 2 2 Object_55 0	Object_root Object_root Object_1 Object_2 Object_3 Object_3 Object_4 Object_5 root_Object root_Object Object_1 Object_55 Object_3 Object_3 Object_15 Object_5	Object_root Object_root Object_1 Object_2 Object_3 Object_3 Object_4 Object_5 root_Object root_Object Object_1 Object_55 Object_3 Object_3 Object_15 Object_5
Object_root Object_root Object_1 Object_root Object_1 Object_root Object_3 Object_root Object_3 Object_7 Object_7 1 1 root 3 1 Object_71 2 5 Object_25 2 1 Object_11 2 3 Object_31 0	Object_root Object_root Object_1 Object_3 root root Object_11 Object_3	Object_root Object_root Object_1 Object_3 root root Object_11 Object_3
object_root object_root obj_1 object_root obj_2 object_root obj_2 object_root obj_3 object_root obj_4 obj_1 obj_2 obj_3 obj_1 obj_4 obj_3 obj_4 obj_5 obj_9 obj_9 1 1 root 2 1 obj_11 2 4 obj_44 2 3 obj_69 2 9 obj_00 3 2 obj_55	object_root object_root obj_1 obj_2 obj_3 obj_4 obj_4 obj_3 obj_5 root root obj_11 obj_2 obj_69 obj_44 obj_44 obj_3 obj_55	object_root object_root obj_1 obj_2 obj_3 obj_4 obj_4 obj_3 obj_5 root root obj_11 obj_2 obj_69 obj_44 obj_44 obj_3 obj_55

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
3 3 obj_67 0		

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.
2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avvora.ru/student/files/methodichescoe_posobie_dlya_laboratornyh_rabot_3.pdf (дата обращения 05.05.2021).
3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avvora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).
4. Шилдт Г. С++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.
5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».
6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).